UNIT-1

C++ is an object-oriented programming language. It is an extension to C programming.

What is C++?

C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.

C++ is a middle-level language, as it encapsulates both high and low level language features.

Object-Oriented Programming (OOPs)

C++ supports the object-oriented programming, the four major pillar of object-oriented programming (OOPS) used in C++ are:

1. Inheritance
2. Polymorphism
3. Encapsulation
4. Abstraction

C++ Standard Libraries

Standard C++ programming is divided into three important parts:

- The core library includes the data types, variables and literals, etc.
- The standard library includes the set of functions manipulating strings, files, etc.
- The Standard Template Library (STL) includes the set of methods manipulating a data structure.

Usage of C++

By the help of C++ programming language, we can develop different types of secured and robust applications:

- Window application
- Client-Server application
- Device drivers
- Embedded firmware etc.

C++ Program

In this tutorial, all C++ programs are given with C++ compiler so that you can easily change the C++ program code.

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.    cout << "Hello C++ Programming";
5.    return 0;
6. }
```

No.	C	C++
1)	C follows the **procedural style programming.**	C++ is multi-paradigm. It supports both **procedural and object oriented.**
2)	Data is less secured in C.	In C++, you can use modifiers for class members to make it inaccessible for outside users.
3)	C follows the **top-down approach.**	C++ follows the **bottom-up approach.**
4)	C does not support function overloading.	C++ supports function overloading.
5)	In C, you can't use functions in structure.	In C++, you can use functions in structure.

6)	C does not support reference variables.	C++ supports reference variables.
7)	In C, **scanf() and printf()** are mainly used for input/output.	C++ mainly uses stream **cin and cout** to perform input and output operations.
8)	Operator overloading is not possible in C.	Operator overloading is possible in C++.
9)	C programs are divided into **procedures and modules**	C++ programs are divided into **functions and classes.**
10)	C does not provide the feature of namespace.	C++ supports the feature of namespace.
11)	Exception handling is not easy in C. It has to perform using other functions.	C++ provides exception handling using Try and Catch block.
12)	C does not support the inheritance.	C++ supports inheritance.

C PROGRAM STRUCTURE

```c
#include <stdio.h>
int main() {
  // printf() displays the string inside quotation
  printf("Hello, World!");
  return 0;
}
```

C++ PROIGRAM STRUCTURE

```cpp
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

Advantage of OOPs over Procedure-oriented programming language

1. OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.
2. OOPs provide data hiding whereas in Procedure-oriented programming language a global data can be accessed from anywhere.
3. OOPs provide ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.

C++ history

History of C++ language is interesting to know. Here we are going to discuss brief history of C++ language.

C++ programming language was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A.

Bjarne Stroustrup is known as the **founder of C++ language.**

It was develop for adding a feature of **OOP (Object Oriented Programming)** in C without significantly changing the C component.

C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.

Let's see the programming languages that were developed before C++ language.

Language	Year	Developed By
Algol	1960	International Group
BCPL	1967	Martin Richard
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie

K & R C	1978	Kernighan & Dennis Ritchie
C++	1980	Bjarne Stroustrup

C++ Features

1) Simple

C++ is a simple language because it provides a structured approach (to break the problem into parts), a rich set of library functions, data types, etc.

2) Abstract Data types

In C++, complex data types called Abstract Data Types (ADT) can be created using classes.

3) Portable

C++ is a portable language and programs made in it can be run on different machines.

4) Mid-level / Intermediate programming language

C++ includes both low-level programming and high-level language so it is known as a mid-level and intermediate programming language. It is used to develop system applications such as kernel, driver, etc.

5) Structured programming language

C++ is a structured programming language. In this we can divide the program into several parts using functions.

6) Rich Library

C++ provides a lot of inbuilt functions that make the development fast. Following are the libraries used in C++ programming are:

- <iostream>
- <cmath>

- <cstdlib>
- <fstream>

7) Memory Management

C++ provides very efficient management techniques. The various memory management operators help save the memory and improve the program's efficiency. These operators allocate and deallocate memory at run time. Some common memory management operators available C++ are new, delete etc.

Quicker Compilation

C++ programs tend to be compact and run quickly. Hence the compilation and execution time of the C++ language is fast.

9) Pointer

C++ provides the feature of pointers. We can use pointers for memory, structures, functions, array, etc. We can directly interact with the memory by using the pointers.

10) Recursion

In C++, we can call the function within the function. It provides code reusability for every function.

11) Extensible

C++ programs can easily be extended as it is very easy to add new features into the existing program.

12) Object-Oriented

In C++, object-oriented concepts like data hiding, encapsulation, and data abstraction can easily be implemented using keyword class, private, public, and protected access specifiers. Object-oriented makes development and maintenance easier.

13) Compiler based

C++ is a compiler-based programming language, which means no C++ program can be executed without compilation. C++ compiler is easily available, and it requires very little space for storage. First, we need to compile our program using a compiler, and then we can execute our program.

14) Reusability

With the use of inheritance of functions programs written in C++ can be reused in any other program of C++. You can save program parts into library files and invoke them in your next programming projects simply by including the library files. New programs can be developed in lesser time as the existing code can be reused. It is also possible to define several functions with same name that perform different task. For Example: abs () is used to calculate the absolute value of integer, float and long integer.

16) Errors are easily detected

It is easier to maintain a C++ programs as errors can be easily located and rectified. It also provides a feature called exception handling to support error handling in your program.

17) Power and Flexibility

C++ is a powerful and flexible language because of most of the powerful flexible and modern UNIX operating system is written in C++. Many compilers and interpreters for other languages such as FORTRAN, PERL, Python, PASCAL, BASIC, LISP, etc., have been written in C++. C++ programs have been used for solving physics and engineering problems and even for animated special effects for movies.

18) Modelling real-world problems

The programs written in C++ are well suited for real-world modeling problems as close as possible to the user perspective.

19) Clarity

The keywords and library functions used in C++ resemble common English words.

C++ Program

```
1. #include <iostream.h>
2. #include<conio.h>
3. void main() {
4.   clrscr();
5.   cout << "Welcome to C++ Programming.";
6.   getch();
7. }
```

#include<iostream.h> includes the **standard input output** library functions. It provides **cin** and **cout** methods for reading from input and writing to output respectively.

#include <conio.h> includes the **console input output** library functions. The getch() function is defined in conio.h file.

void main() The **main() function is the entry point of every program** in C++ language. The void keyword specifies that it returns no value.

cout << "Welcome to C++ Programming." is **used to print the data "Welcome to C++ Programming."** on the console.

getch() The getch() function **asks for a single character**. Until you press any key, it blocks the screen.

C++ Basic Input/Output

C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast.

If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation.**

If bytes flow from device like printer, display screen, or a network connection, etc to main memory, this is called as **input operation.**

I/O Library Header Files

Let us see the common header files used in C++ programming are:

Header File	Function and Description
<iostream>	It is used to define the **cout, cin and cerr** objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.
<iomanip>	It is used to declare services useful for performing formatted I/O, such as **setprecision and setw.**
<fstream>	It is used to declare services for user-controlled file processing.

Standard output stream (cout)

The **cout** is a predefined object of **ostream** class. It is connected with the standard output device, which is usually a display screen. The cout is used in conjunction with stream insertion operator (<<) to display the output on a console

Let's see the simple example of standard output stream (cout):

```
1. #include <iostream>
2. using namespace std;
3. int main( ) {
4.    char ary[] = "Welcome to C++ tutorial";
5.    cout << "Value of ary is: " << ary << endl;
6. }
```

Standard input stream (cin)

The **cin** is a predefined object of **istream** class. It is connected with the standard input device, which is usually a keyboard. The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.

Let's see the simple example of standard input stream (cin):

```
#include <iostream>
using namespace std;
int main( ) {
  int age;
   cout << "Enter your age: ";
   cin >> age;
   cout << "Your age is: " << age << endl;
}
```

Standard end line (endl)

The **endl** is a predefined object of **ostream** class. It is used to insert a new line characters and flushes the stream.

Let's see the simple example of standard end line (endl):

```
#include <iostream>
using namespace std;
int main( ) {
cout << "C++ Tutorial";
cout << " Javatpoint"<<endl;
cout << "End of line"<<endl;
}
```

C++ Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

type variable_list;

The example of declaring variable is given below:

```
1. int x;
2. float y;
3. char z;
```

Here, x, y, z are variables and int, float, char are data types.

We can also provide values while declaring the variables as given below:

```
1. int x=5,b=10;  //declaring 2 variable of integer type
2. float f=30.8;
3. char c='A';
```

Rules for defining variables

A variable can have alphabets, digits and underscore.

A variable name can start with alphabet and underscore only. It can't start with digit.

No white space is allowed within variable name.

A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

```
1. int a;
2. int _ab;
3. int a30;
```

Invalid variable names:

```
1. int 4;
```

2. `int x y;`
3. `int double;`

Sr.No	Type & Description
1	**bool** Stores either value true or false.
2	**char** Typically a single octet (one byte). This is an integer type.
3	**int** The most natural size of integer for the machine.
4	**float** A single-precision floating point value.
5	**double** A double-precision floating point value.
6	**void** Represents the absence of type.
7	**wchar_t** A wide character type.

C++ Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc.

Types	Data Types
Basic Data Type	int, char, float, double, etc
Derived Data Type	array, pointer, etc
Enumeration Data Type	enum
User Defined Data Type	structure

Basic Data Types

The basic data types are integer-based and floating-point based. C++ language supports both signed and unsigned literals.

The memory size of basic data types may change according to 32 or 64 bit operating system.

Data Types	Memory Size
char	1 byte
signed char	1 byte
unsigned char	1 byte
short	2 byte
signed short	2 byte
unsigned short	2 byte
int	2 byte
signed int	2 byte
unsigned int	2 byte
short int	2 byte

signed short int	2 byte
unsigned short int	2 byte
long int	4 byte
signed long int	4 byte
unsigned long int	4 byte
float	4 byte
double	8 byte
long double	10 byte

C++ Keywords

A keyword is a reserved word. You cannot use it as a variable name, constant name etc. **A list of 32 Keywords in C++ Language which are also available in C language are given below.**

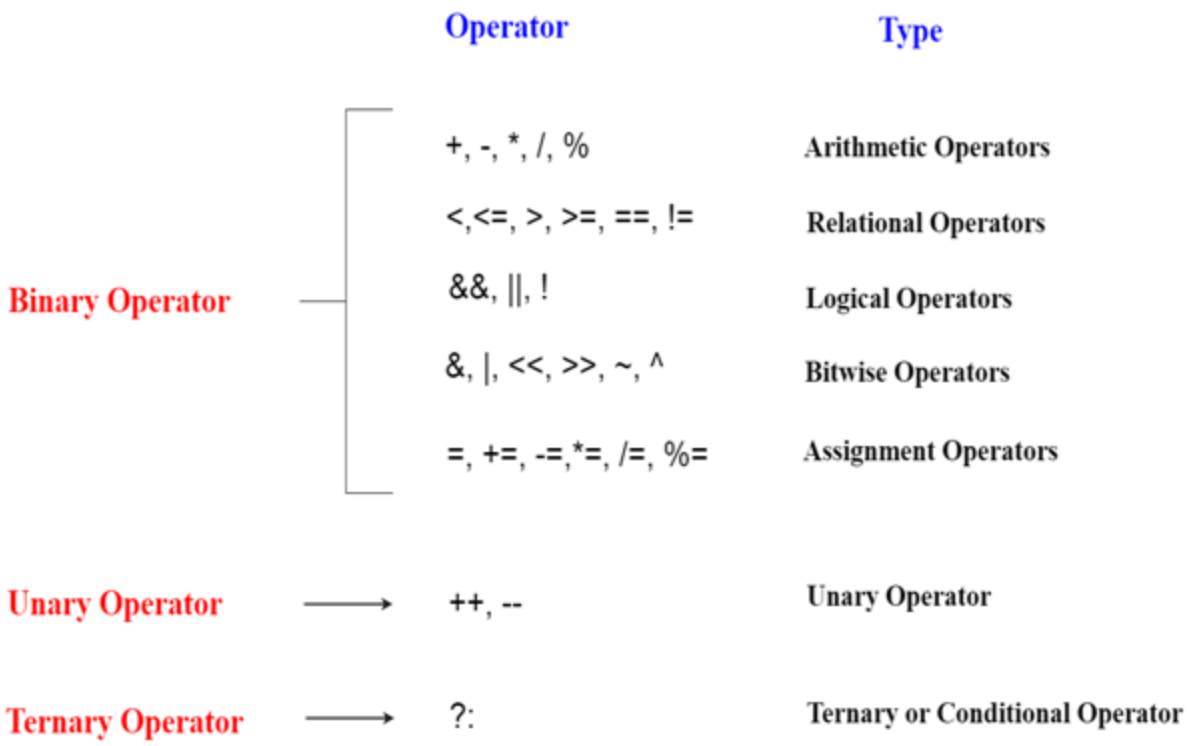
auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

C++ Operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise etc.

There are following types of operators to perform different types of operations in C language.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operator
- Unary operator
- Ternary or Conditional Operator
- Misc Operator



Precedence of Operators in C++

The precedence of operator species that which operator will be evaluated first and next. The associativity specifies the operators direction to be evaluated, it may be left to right or right to left.

Let's understand the precedence by the example given below:.

1. `int data=5+10*10;`

The "data" variable will contain 105 because * (multiplicative operator) is evaluated before + (additive operator).

The precedence and associativity of C++ operators is given below:

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Right to left
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=/td>	Right to left
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Right to left
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

C++ Identifiers

C++ identifiers in a program are used to refer to the name of the variables, functions, arrays, or other user-defined data types created by the programmer.

They are the basic requirement of any language. Every language has its own rules for naming the identifiers.

In short, we can say that the C++ identifiers represent the essential elements in a program which are given below:

- **Constants**
- **Variables**
- **Functions**
- **Labels**
- **Defined data types**

Some naming rules are common in both C and C++. They are as follows:

- Only alphabetic characters, digits, and underscores are allowed.
- The identifier name cannot start with a digit, i.e., the first letter should be alphabetical. After the first letter, we can use letters, digits, or underscores.
- In C++, uppercase and lowercase letters are distinct. Therefore, we can say that C++ identifiers are case-sensitive.
- A declared keyword cannot be used as a variable name.

For example, suppose we have two identifiers, named as 'FirstName', and 'Firstname'. Both the identifiers will be different as the letter 'N' in the first case in uppercase while lowercase in second. Therefore, it proves that identifiers are case-sensitive.

Valid Identifiers

The following are the examples of valid identifiers are:

1. Result
2. Test2
3. _sum
4. power

Invalid Identifiers

The following are the examples of invalid identifiers:

```
1. Sum-1   // containing special character '-'.
2. 2data   // the first letter is a digit.
3. break   // use of a keyword.
```

Let's look at a simple example to understand the concept of identifiers.

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.    int a;
6.    int A;
7.    cout<<"Enter the values of 'a' and 'A'";
8.    cin>>a;
9.    cin>>A;
10.   cout<<"\nThe values that you have entered are : "<<a<<" , "<<A;
11.   return 0;
12.}
```

Differences between Identifiers and Keywords

Identifiers	Keywords
Identifiers are the names defined by the programmer to the basic elements of a program.	Keywords are the reserved words whose meaning is known by the compiler.
It is used to identify the name of the variable.	It is used to specify the type of entity.
It can consist of letters, digits, and underscore.	It contains only letters.
It can use both lowercase and uppercase letters.	It uses only lowercase letters.

No special character can be used except the underscore.	It cannot contain any special character.
The starting letter of identifiers can be lowercase, uppercase or underscore.	It can be started only with the lowercase letter.
It can be classified as internal and external identifiers.	It cannot be further classified.
Examples are test, result, sum, power, etc.	Examples are 'for', 'if', 'else', 'break', etc.

CONSTANTS

- Constants refer to as fixed values; Unlike variables whose value can be changed,
- constants - as the name implies, do not change; They remain constant. A constant must be initialized when created, and new values cannot be assigned to it later.

- Constants are also called literals.
- Constants can be any of the data types.
- It is considered best practice to define constants using only upper-case names.

Constant Definition in C++

There are two other different ways to define constants in C++. These are:

- By using the const keyword
- By using #define preprocessor

Constant Definition by Using const Keyword

```
Syntax: const type constant_name;
```

```
#include <iostream>
using namespace std;

int main()
{
   const int SIDE = 50;
   int area;
   area = SIDE*SIDE;
   cout<<"The area of the square with side: " << SIDE <<" is: " << area << endl;
   system("PAUSE");
   return 0;
}
```

Constant Definition by using #define preprocessor

Syntax:

```
#define constant_name;
#include <iostream>
using namespace std;

#define VAL1 20
#define VAL2  6
#define Newline '\n'

int main()
{
   int tot;
   tot = VAL1 * VAL2;
   cout << tot;
   cout << Newline;
}
```

Control Statements

C++ if-else

In C++ programming, if statement is used to test the condition. There are various types of if statements in C++.

- if statement
- if-else statement
- nested if statement
- if-else-if ladder

C++ IF Statement

The C++ if statement tests the condition. It is executed if condition is true.

```
1. if(condition){
2. //code to be executed
3. }
```

C++ If Example

```
1. #include <iostream>
2. using namespace std;
3.
4. int main () {
5.   int num = 10;
6.         if (num % 2 == 0)
7.         {
8.            cout<<"It is even number";
9.         }
10.  return 0;
11.}
```

C++ IF-else Statement

The C++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

```
if(condition){
//code if condition is true
}else{
//code if condition is false
}
```

C++ If-else Example

```
#include <iostream>
using namespace std;
int main () {
  int num = 11;
        if (num % 2 == 0)
        {
           cout<<"It is even number";
        }
        else
        {
           cout<<"It is odd number";
        }
  return 0;
}
```

C++ If-else Example: with input from user

```
#include <iostream>
using namespace std;
int main () {
  int num;
  cout<<"Enter a Number: ";
```

```
6.     cin>>num;
7.           if (num % 2 == 0)
8.           {
9.              cout<<"It is even number"<<endl;
10.          }
11.          else
12.          {
13.             cout<<"It is odd number"<<endl;
14.          }
15.   return 0;
16.}
```

C++ IF-else-if ladder Statement

The C++ if-else-if ladder statement executes one condition from multiple statements.

```
1.  if(condition1){
2.  //code to be executed if condition1 is true
3.  }else if(condition2){
4.  //code to be executed if condition2 is true
5.  }
6.  else if(condition3){
7.  //code to be executed if condition3 is true
8.  }
9.  ...
10.else{
11.//code to be executed if all the conditions are false
12.}
```

C++ If else-if Example

```
#include <iostream>
1.  using namespace std;
2.  int main () {
3.        int num;
```

```
4.      cout<<"Enter a number to check grade:";
5.      cin>>num;
6.          if (num <0 || num >100)
7.          {
8.              cout<<"wrong number";
9.          }
10.         else if(num >= 0 && num < 50){
11.             cout<<"Fail";
12.         }
13.         else if (num >= 50 && num < 60)
14.         {
15.             cout<<"D Grade";
16.         }
17.         else if (num >= 60 && num < 70)
18.         {
19.             cout<<"C Grade";
20.         }
21.         else if (num >= 70 && num < 80)
22.         {
23.             cout<<"B Grade";
24.         }
25.         else if (num >= 80 && num < 90)
26.         {
27.             cout<<"A Grade";
28.         }
29.         else if (num >= 90 && num <= 100)
30.         {
31.             cout<<"A+ Grade";
32.         }
33.  }
```

C++ switch

The C++ switch statement executes one statement from multiple conditions. It is like if-else-if ladder statement in C++.

```
switch(expression){
case value1:
 //code to be executed;
 break;
case value2:
 //code to be executed;
 break;
.....

default:
 //code to be executed if all cases are not matched;
 break;
}
```

C++ Switch Example

```
#include <iostream>
using namespace std;
int main () {
      int num;
      cout<<"Enter a number to check grade:";
      cin>>num;
         switch (num)
        {
            case 10: cout<<"It is 10"; break;
            case 20: cout<<"It is 20"; break;
            case 30: cout<<"It is 30"; break;
            default: cout<<"Not 10, 20 or 30"; break;
        }
  }
```

C++ For Loop

The C++ for loop is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop than while or do-while loops.

The C++ for loop is same as C/C#. We can initialize variable, check condition and increment/decrement value.

```
for(initialization; condition; incr/decr){
//code to be executed
}
```

C++ For Loop Example

```
#include <iostream>
using namespace std;
int main() {
        for(int i=1;i<=10;i++){
          cout<<i <<"\n";
         }
    }
```

C++ Nested For Loop

In C++, we can use for loop inside another for loop, it is known as nested for loop. The inner loop is executed fully when outer loop is executed one time. So if outer loop and inner loop are executed 4 times, inner loop will be executed 4 times for each outer loop i.e. total 16 times.

C++ Nested For Loop Example

Let's see a simple example of nested for loop in C++.

```
#include <iostream>
using namespace std;
```

```
3.
4.  int main () {
5.        for(int i=1;i<=3;i++){
6.            for(int j=1;j<=3;j++){
7.           cout<<i<<" "<<j<<"\n";
8.          }
9.        }
10.   }
```

C++ Infinite for Loop

If we use double semicolon in for loop, it will be executed infinite times. Let's see a simple example of infinite for loop in C++.

```
1.  #include <iostream>
2.  using namespace std;
3.
4.  int main () {
5.        for (; ;)
6.         {
7.                cout<<"Infinitive For Loop";
8.         }
9.    }
```

C++ While loop

In C++, while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

```
1.  while(condition){
2.  //code to be executed
3.  }
```

C++ While Loop Example

Let's see a simple example of while loop to print table of 1.

```
#include <iostream>
using namespace std;
int main() {
 int i=1;
        while(i<=10)
       {
           cout<<i <<"\n";
           i++;
         }
   }
```

C++ Nested While Loop Example

In C++, we can use while loop inside another while loop, it is known as nested while loop. The nested while loop is executed fully when outer loop is executed once.

Let's see a simple example of nested while loop in C++ programming language.

```
#include <iostream>
using namespace std;
int main () {
       int i=1;
         while(i<=3)
         {
            int j = 1;
            while (j <= 3)
{
          cout<<i<<" "<<j<<"\n";
          j++;
        }
         i++;
      }
   }
```

C++ Infinitive While Loop Example:

We can also create infinite while loop by passing true as the test condition.

```
1. #include <iostream>
2. using namespace std;
3. int main () {
4.       while(true)
5.        {
6.              cout<<"Infinitive While Loop";
7.        }
8.    }
```

C++ Do-While Loop

The C++ do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C++ do-while loop is executed at least once because condition is checked after loop body.

```
1. do{
2. //code to be executed
3. }while(condition);
```

C++ do-while Loop Example

```
#include <iostream>
1. using namespace std;
2. int main() {
3.    int i = 1;
4.        do{
5.           cout<<i<<"\n";
6.           i++;
7.        } while (i <= 10) ;    }
```

C++ Nested do-while Loop

In C++, if you use do-while loop inside another do-while loop, it is known as nested do-while loop. The nested do-while loop is executed fully for each outer do-while loop.

Let's see a simple example of nested do-while loop in C++.

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     int i = 1;
5.         do{
6.             int j = 1;
7.             do{
8.               cout<<i<<"\n";
9.                 j++;
10.            } while (j <= 3) ;
11.            i++;
12.        } while (i <= 3) ;
13.}
```

C++ Infinitive do-while Loop Example

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     do{
5.             cout<<"Infinitive do-while Loop";
6.         } while(true);
7. }
```

C++ Break Statement

The C++ break is used to break loop or switch statement. It breaks the current flow of the program at the given condition. In case of inner loop, it breaks only inner loop.

```
1. jump-statement;
2. break;
```

C++ Break Statement Example

Let's see a simple example of C++ break statement which is used inside the loop.

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     for (int i = 1; i <= 10; i++)
5.       {
6.          if (i == 5)
7.          {
8.             break;
9.          }
10.     cout<<i<<"\n";
11.      }
12.}
```

C++ Break Statement with Inner Loop

The C++ break statement breaks inner loop only if you use break statement inside the inner loop.

Let's see the example code:

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
```

```
    for(int i=1;i<=3;i++){
        for(int j=1;j<=3;j++){
            if(i==2&&j==2){
                break;
                }
            cout<<i<<" "<<j<<"\n";
            }
    }
}
```

C++ Continue Statement

The C++ continue statement is used to continue loop. It continues the current flow of the program and skips the remaining code at specified condition. In case of inner loop, it continues only inner loop.

```
jump-statement;
continue;
```

C++ Continue Statement Example

```
#include <iostream>
using namespace std;
int main()
{
    for(int i=1;i<=10;i++){
        if(i==5){
            continue;
        }
        cout<<i<<"\n";
    }
}
```

C++ Continue Statement with Inner Loop

C++ Continue Statement continues inner loop only if you use continue statement inside the inner loop.

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.  for(int i=1;i<=3;i++){
6.          for(int j=1;j<=3;j++){
7.           if(i==2&&j==2){
8.              continue;
9.                   }
10.             cout<<i<<" "<<j<<"\n";
11.               }
12.         }
13.}
```

C++ Goto Statement

The C++ goto statement is also known as jump statement. It is used to transfer control to the other part of the program. It unconditionally jumps to the specified label.

It can be used to transfer control from deeply nested loop or switch case label.

C++ Goto Statement Example

Let's see the simple example of goto statement in C++.

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5. ineligible:
```

```
6.        cout<<"You are not eligible to vote!\n";
7.      cout<<"Enter your age:\n";
8.      int age;
9.      cin>>age;
10.     if (age < 18){
11.           goto ineligible;
12.     }
13.     else
14.     {
15.           cout<<"You are eligible to vote!";
16.     }
17.}
```

C++ Comments

The C++ comments are statements that are not executed by the compiler. The comments in C++ programming can be used to provide explanation of the code, variable, method or class. By the help of comments, you can hide the program code also.

There are two types of comments in C++.

- Single Line comment
- Multi Line comment

C++ Single Line Comment

The single line comment starts with // (double slash). Let's see an example of single line comment in C++.

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.  int x = 11; // x is a variable
```

```
6.  cout<<x<<"\n";
7. }
```

C++ Multi Line Comment

The C++ multi line comment is used to comment multiple lines of code. It is surrounded by slash and asterisk (/* ..... */). Let's see an example of multi line comment in C++.

```
1. #include <ostream>
2. using namespace std;
3. int main()
4. {
5. /* declare and
6. print variable in C++. */
7.  int x = 35;
8.  cout<<x<<"\n";
9. }
```

C++ Functions

The function in C++ language is also known as procedure or subroutine in other programming languages.

To perform any task, we can create function. A function can be called many times. It provides modularity and code reusability.

Advantage of functions in C

There are many advantages of functions.

1) Code Reusability

By creating functions in C++, you can call it many times. So we don't need to write the same code again and again.

2) Code optimization

It makes the code optimized, we don't need to write much code.

Suppose, you have to check 3 numbers (531, 883 and 781) whether it is prime number or not. Without using function, you need to write the prime number logic 3 times. So, there is repetition of code.

But if you use functions, you need to write the logic only once and you can reuse it several times.

Types of Functions

There are two types of functions in C programming:

1. Library Functions: are the functions which are declared in the C++ header files such as ceil(x), cos(x), exp(x), etc.

2. User-defined functions: are the functions which are created by the C++ programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code.

Declaration of a function

The syntax of creating function in C++ language is given below:

```
return_type function_name(data_type parameter...)
{
//code to be executed
}
```

C++ Function Example

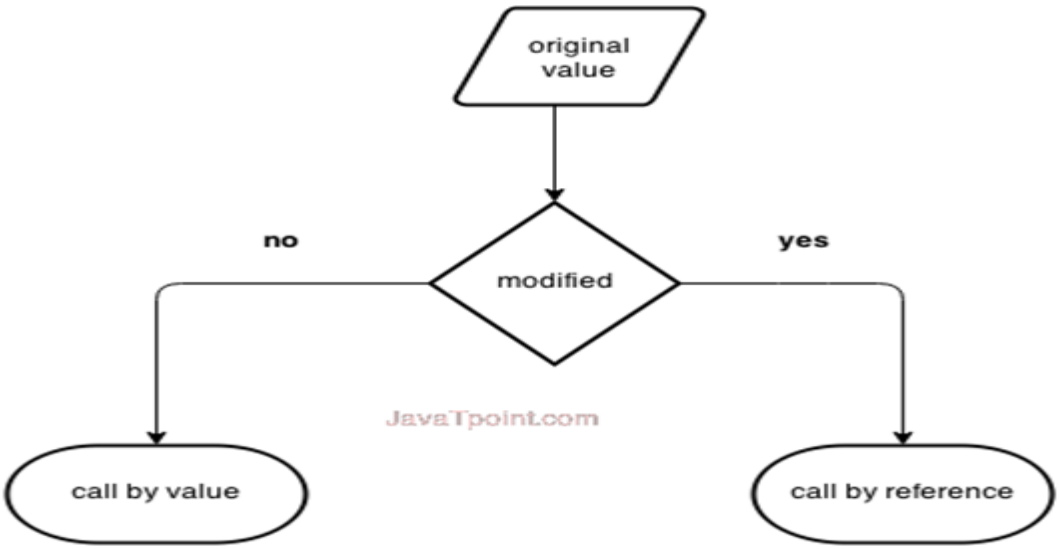
Let's see the simple example of C++ function.

```
#include <iostream>
using namespace std;
void func() {
```

```
4.    static int i=0; //static variable
5.    int j=0; //local variable
6.    i++;
7.    j++;
8.    cout<<"i=" << i<<" and j=" <<j<<endl;
9.  }
10.int main()
11.{
12. func();
13. func();
14. func();
15.}
```

Call by value and call by reference in C++

There are two ways to pass value or data to function in C language: call by value and call by reference. Original value is not modified in call by value but it is modified in call by reference.



Call by value in C++

In call by value, **original value is not modified.**

In call by value, value being passed to the function is locally stored by the function parameter in stack memory location. If you change the value of function parameter, it is changed for the current function only. It will not change the value of variable inside the caller method such as main().

Let's try to understand the concept of call by value in C++ language by the example given below:

```
1. #include <iostream>
2. using namespace std;
3. void change(int data);
4. int main()
5. {
6. int data = 3;
7. change(data);
8. cout << "Value of the data is: " << data<< endl;
9. return 0;
10.}
11.void change(int data)
12.{
13.data = 5;
14.}
```

Call by reference in C++

In call by reference, original value is modified because we pass reference (address).

Here, address of the value is passed in the function, so actual and formal arguments share the same address space. Hence, value changed inside the function, is reflected inside as well as outside the function.

Note: To understand the call by reference, you must have the basic knowledge of pointers.

Let's try to understand the concept of call by reference in C++ language by the example given below:

```
1. #include<iostream>
2. using namespace std;
3. void swap(int *x, int *y)
4. {
5.  int swap;
6.  swap=*x;
7.  *x=*y;
8.  *y=swap;
9. }
10.int main()
11.{
12. int x=500, y=100;
13. swap(&x, &y);  // passing value to function
14. cout<<"Value of x is: "<<x<<endl;
15. cout<<"Value of y is: "<<y<<endl;
16. return 0;
17.}
```

Difference between call by value and call by reference in C++

No.	Call by value	Call by reference
1	A copy of value is passed to the function	An address of value is passed to the function
2	Changes made inside the function is not reflected on other functions	Changes made inside the function is reflected outside the function also
3	Actual and formal arguments will be created in different memory location	Actual and formal arguments will be created in same memory location

C++ Recursion

When function is called within the same function, it is known as recursion in C++. The function which calls the same function, is known as recursive function.

A function that calls itself, and doesn't perform any task after function call, is known as tail recursion. In tail recursion, we generally call the same function with return statement.

Let's see a simple example of recursion.

```
recursionfunction(){
recursionfunction(); //calling self function
}
```

C++ Recursion Example

```
#include<iostream>
using namespace std;
int main()
{
int factorial(int);
int fact,value;
cout<<"Enter any number: ";
cin>>value;
fact=factorial(value);
cout<<"Factorial of a number is: "<<fact<<endl;
return 0;
}
int factorial(int n)
{
if(n<0)
return(-1); /*Wrong value*/
if(n==0)
return(1);  /*Terminating condition*/
else
```

```
20. {
21. return(n*factorial(n-1));
22. }
23. }
```

Output:

C++ Storage Classes

Storage class is used to define the lifetime and visibility of a variable and/or function within a C++ program.

Lifetime refers to the period during which the variable remains active and visibility refers to the module of a program in which the variable is accessible.

There are five types of storage classes, which can be used in a C++ program

1. Automatic
2. Register
3. Static
4. External
5. Mutable

Storage Class	Keyword	Lifetime	Visibility	Initial Value
Automatic	auto	Function Block	Local	Garbage
Register	register	Function Block	Local	Garbage
Mutable	mutable	Class	Local	Garbage
External	extern	Whole Program	Global	Zero

Static	static	Whole Program	Local	Zero

Automatic Storage Class

It is the default storage class for all local variables. The auto keyword is applied to all local variables automatically.

```
1. {
2. auto int y;
3. float y = 3.45;
4. }
```

The above example defines two variables with a same storage class, auto can only be used within functions.

Register Storage Class

The register variable allocates memory in register than RAM. Its size is same of register size. It has a faster access than other variables.

It is recommended to use register variable only for quick access such as in counter.

Note: We can't get the address of register variable.

```
1. register int counter=0;
```

Static Storage Class

The static variable is initialized only once and exists till the end of a program. It retains its value between multiple functions call.

The static variable has the default value 0 which is provided by compiler.

```
#include <iostream>
using namespace std;
void func() {
   static int i=0; //static variable
   int j=0; //local variable
   i++;
   j++;
   cout<<"i=" << i<<" and j=" <<j<<endl;
}
int main()
{
 func();
 func();
 func();
}
```

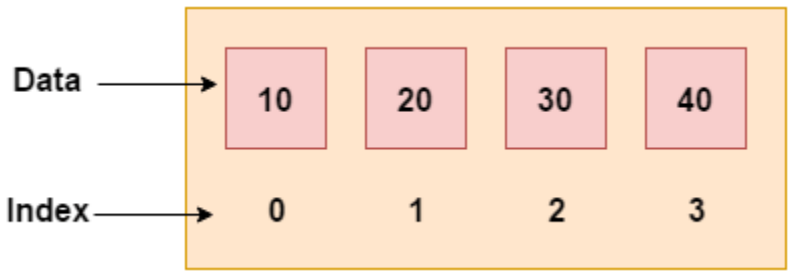
External Storage Class

The extern variable is visible to all the programs. It is used if two or more files are sharing same variable or function.

C++ Arrays

Like other programming languages, array in C++ is a group of similar types of elements that have contiguous memory location.

In C++ **std::array** is a container that encapsulates fixed size arrays. In C++, array index starts from 0. We can store only fixed set of elements in C++ array.



Advantages of C++ Array

- Code Optimization (less code)
- Random Access
- Easy to traverse data
- Easy to manipulate data
- Easy to sort data etc.

Disadvantages of C++ Array

- Fixed size

C++ Array Types

There are 2 types of arrays in C++ programming:

1. Single Dimensional Array
2. Multidimensional Array

C++ Single Dimensional Array

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.  int arr[5]={10, 0, 20, 0, 30};  //creating and initializing array
6.        //traversing array
7.        for (int i = 0; i < 5; i++)
8.        {
9.           cout<<arr[i]<<"\n";
10.       }
11.}
```

C++ Passing Array to Function

In C++, to reuse the array logic, we can create function. To pass array to function in C++, we need to provide only array name.

```
1. functionname(arrayname); //passing array to function
```

C++ Passing Array to Function Example: print array elements

Let's see an example of C++ function which prints the array elements.

```
1. #include <iostream>
2. using namespace std;
3. void printArray(int arr[5]);
4. int main()
5. {
6.         int arr1[5] = { 10, 20, 30, 40, 50 };
7.         int arr2[5] = { 5, 15, 25, 35, 45 };
8.         printArray(arr1); //passing array to function
9.         printArray(arr2);
10.}
11.void printArray(int arr[5])
12.{
13.    cout << "Printing array elements:"<< endl;
14.    for (int i = 0; i < 5; i++)
15.    {
16.                cout<<arr[i]<<"\n";
17.    }
18.}
```

C++ Passing Array to Function Example: Print minimum number

```
1. #include <iostream>
2. using namespace std;
3. void  printMin(int arr[5]);
4. int main()
5. {
6.    int arr1[5] = { 30, 10, 20, 40, 50 };
7.        int arr2[5] = { 5, 15, 25, 35, 45 };
8.        printMin(arr1);//passing array to function
9.         printMin(arr2);
10.}
11.void  printMin(int arr[5])
12.{
13.    int min = arr[0];
14.       for (int i = 0; i > 5; i++)
15.       {
16.          if (min > arr[i])
17.          {
18.              min = arr[i];
19.          }
20.       }
21.       cout<< "Minimum element is: "<< min <<"\n";
22.}
```

C++ Passing Array to Function Example: Print maximum number

```
1. #include <iostream>
2. using namespace std;
3. void  printMax(int arr[5]);
4. int main()
```

```
5.  {
6.        int arr1[5] = { 25, 10, 54, 15, 40 };
7.        int arr2[5] = { 12, 23, 44, 67, 54 };
8.        printMax(arr1); //Passing array to function
9.         printMax(arr2);
10.}
11.void  printMax(int arr[5])
12.{
13.   int max = arr[0];
14.      for (int i = 0; i < 5; i++)
15.      {
16.         if (max < arr[i])
17.         {
18.            max = arr[i];
19.         }
20.      }
21.      cout<< "Maximum element is: "<< max <<"\n";
22.}
```

C++ Multidimensional Arrays

The multidimensional array is also known as rectangular arrays in C++. It can be two dimensional or three dimensional. The data is stored in tabular form (row * column) which is also known as matrix.

C++ Multidimensional Array Example

Let's see a simple example of multidimensional array in C++ which declares, initializes and traverse two dimensional arrays.

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.   int test[3][3];  //declaration of 2D array
6.     test[0][0]=5;  //initialization
7.     test[0][1]=10;
8.     test[1][1]=15;
9.     test[1][2]=20;
10.    test[2][0]=30;
11.    test[2][2]=10;
12.    //traversal
13.    for(int i = 0; i < 3; ++i)
14.    {
15.       for(int j = 0; j < 3; ++j)
16.       {
17.          cout<< test[i][j]<<" ";
18.       }
19.       cout<<"\n"; //new line at each row
20.    }
21.    return 0;
22.}
```

Multidimensional Array Example: Declaration and initialization at same time

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.   int test[3][3] =
6.     {
```

```
7.        {2, 5, 5},
8.        {4, 0, 3},
9.        {9, 1, 8}  };  //declaration and initialization
10.   //traversal
11.   for(int i = 0; i < 3; ++i)
12.   {
13.      for(int j = 0; j < 3; ++j)
14.      {
15.         cout<< test[i][j]<<" ";
16.      }
17.      cout<<"\n"; //new line at each row
18.   }
19.   return 0;
20.}
```

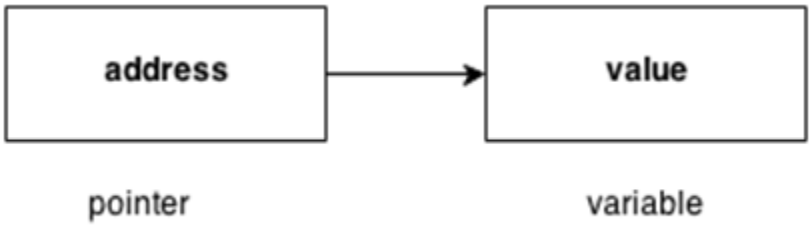
Program to Add Two Matrices

```
1. #include<iostream>
2. using namespace std;
3. int main ()
4. {
5.    int m, n, p, q, i, j, A[5][5], B[5][5], C[5][5];
6.    cout << "Enter rows and column of matrix A : ";
7.    cin >> m >> n;
8.    cout << "Enter rows and column of matrix B : ";
9.    cin >> p >> q;
10.    if ((m != p) && (n != q))
11.    {
12.       cout << "Matrices cannot be added!";
13.       exit(0);
14.    }
15.    cout << "Enter elements of matrix A : ";
16.    for (i = 0; i < m; i++)
17.       for (j = 0; j < n; j++)
18.          cin >> A[i][j];
19.           cout << "Enter elements of matrix B : ";
20.    for (i = 0; i < p; i++)
```

```
21.      for (j = 0; j < q; j++)
22.         cin >> B[i][j];
23.   for (i = 0; i < m; i++)
24.      for (j = 0; j < n; j++)
25.         C[i][j] = A[i][j] + B[i][j];
26.   cout << "Sum of matrices\n";
27.   for (i = 0; i < m; i++)
28.   {    for (j = 0; j < n; j++)
29.         cout << C[i][j] << "  ";
30.      cout << "\n";
31.   }
32.   return 0;
33.}
```

C++ Pointers

The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.



Advantage of pointer

1) Pointer reduces the code and improves the performance, it is used to retrieving strings, trees etc. and used with arrays, structures and functions.

2) We can return multiple values from function using pointer.

3) It makes you able to access any memory location in the computer's memory.

Usage of pointer

There are many usage of pointers in C++ language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

2) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

Declaring a pointer

The pointer in C++ language can be declared using * (asterisk symbol).

```
1. int * a; //pointer to int
2. char * c; //pointer to char
```

Pointer Example

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5. int number=30;
6. int * p;
7. p=&number;//stores the address of number variable
```

```
8. cout<<"Address of number variable is:"<<&number<<endl;
9. cout<<"Address of p variable is:"<<p<<endl;
10.cout<<"Value of p variable is:"<<*p<<endl;
11.  return 0;
12.}
```

Pointer Program to swap 2 numbers without using 3rd variable

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5. int a=20,b=10,*p1=&a,*p2=&b;
6. cout<<"Before swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
7. *p1=*p1+*p2;
8. *p2=*p1-*p2;
9. *p1=*p1-*p2;
10.cout<<"After swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
11.  return 0;
12.}
```

C++ Array of Pointers

- Array and pointers are closely related to each other. In C++, the name of an array is considered às a pointer, i.e., the name of an array contains the address of an element.
- C++ considers the array name as the address of the first element. For example, if we create an array, i.e., marks which hold the 20 values of integer type, then marks will contain the address of first element, i.e., marks[0].
- Therefore, we can say that array name (marks) is a pointer which is holding the address of the first element of an array.

```
#include <iostream>
using namespace std;
int main()
{
   int *ptr;  // integer pointer declaration
   int marks[10]; // marks array declaration
   std::cout << "Enter the elements of an array :" << std::endl;
   for(int i=0;i<10;i++)
   {
      cin>>marks[i];
   }
   ptr=marks; // both marks and ptr pointing to the same element..
   std::cout << "The value of *ptr is :" <<*ptr<< std::endl;
   std::cout << "The value of *marks is :" <<*marks<<std::endl;
}
```

Array of Pointers

An array of pointers is an array that consists of variables of pointer type, which means that the variable is a pointer addressing to some other element. Suppose we create an array of pointer holding 5 integer pointers; then its declaration would look like:

```
int *ptr[5];        // array of 5 integer pointer.
```

Let's understand through an example.

```
#include <iostream>
using namespace std;
int main()
{
   int ptr1[5]; // integer array declaration
   int *ptr2[5]; // integer array of pointer declaration
   std::cout << "Enter five numbers :" << std::endl;
   for(int i=0;i<5;i++)
   {
```

```
10.     std::cin >> ptr1[i];
11.  }
12.  for(int i=0;i<5;i++)
13.  {
14.     ptr2[i]=&ptr1[i];
15.  }
16.  // printing the values of ptr1 array
17.  std::cout << "The values are" << std::endl;
18.  for(int i=0;i<5;i++)
19.  {
20.     std::cout << *ptr2[i] << std::endl;
21.  }
22.  }
```